

上海交通大学生成式人工智能AIGC微专业

机器学习与深度学习基础

欢迎
Welcome

上海交通大学学生生成式人工智能AIGC微专业
机器学习与深度学习基础

Python 编程语言入门

Introduction to Python Programming Language

学生创新中心 赵伟明

Student Innovation Center - Kevin ZHAO Weiming

Python 编程语言入门 - 课程内容

Introduction to Python - Outlines

- Python 语言介绍
- Jupyter Notebook 的使用
- 算术操作与逻辑操作
- 变量
- 数据类型
 - 基本数据类型：数值类型（整型、浮点型）、布尔型、字符型、字符串
 - 高级数据类型：列表、元组、字典
- 字符串操作
- 控制语句：条件语句、循环语句
- 函数：抽象与封装、全局和局部变量
- 类与对象
- 用 Python 读写文件
- 一些函数示例

机器学习与深度学习基础

Python 简介

Introduction to Python programming language



Python 简介

Introduction to Python Programming Language

- Python是一种广泛使用的**解释型**、**高级**和**通用**的编程语言。
- **吉多·范罗苏姆 (Guido van Rossum)** 于1980年代后期开始研发Python，于1991年首次发布 Python 0.9.0。
- Python 2.0于2000 年发布并引入了新功能；Python 3.0于2008年发布，它是该语言的主要修订版，并非完全向后兼容。
- Python的设计哲学，强调代码的**可读性**和**简洁的语法**，尤其是使用空格缩进来划分代码块。相比于C语言或Java，**Python让开发者能够用更少的代码表达想法**。

```
print ("Hello, world!")
```



范罗苏姆在2006年O'Reilly开源大会上

机器学习与深度学习基础

Jupyter Notebook 的使用

Using a Jupyter Notebook

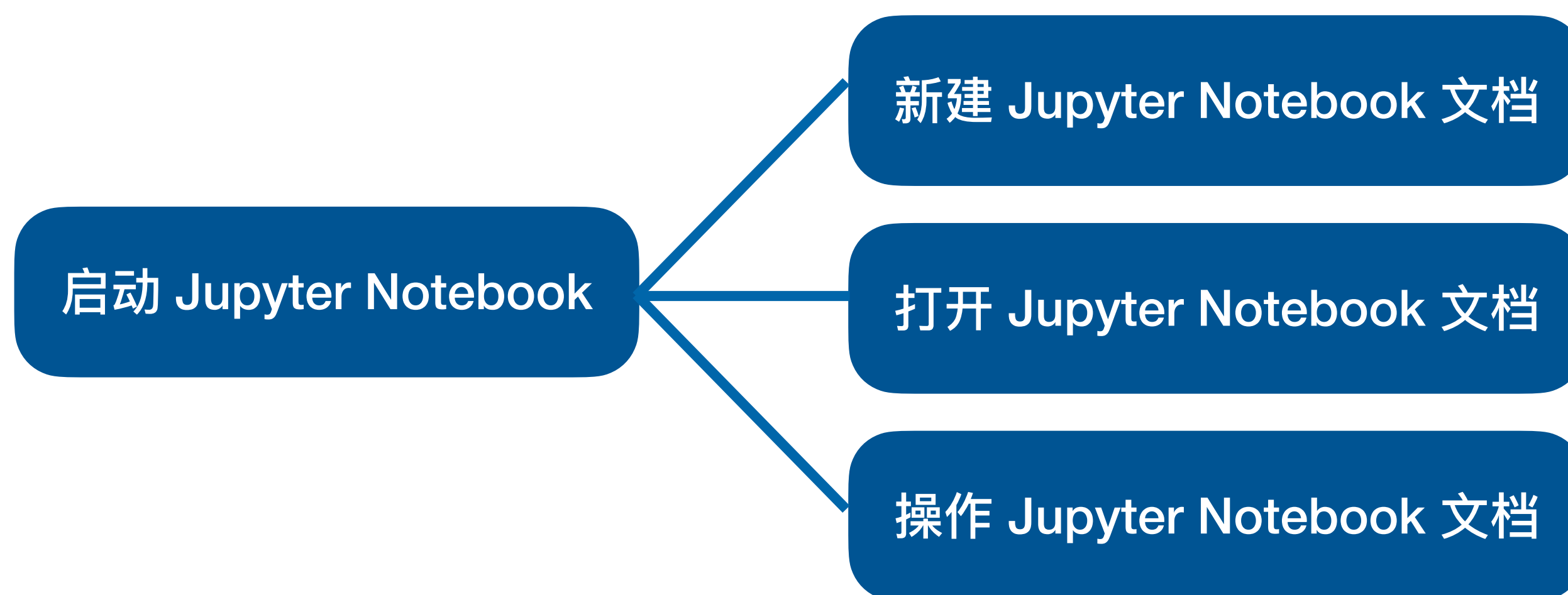


Jupyter Notebook 的使用



Using a Jupyter Notebook

- Jupyter Notebook（前身是IPython Notebook）是一个基于Web的交互式计算环境，用于创建Jupyter Notebook文档。
- Jupyter Notebook文档是一个JSON文档，包含一个有序的输入/输出单元格列表，这些单元格可以包含代码、文本（使用Markdown语言）、数学、图表和富媒体 (Rich media)，通常以“.ipynb”后缀结尾。



启动 Jupyter Notebook



Starting Jupyter Notebook service

- 在终端中加载 base 环境 (Linux)

```
~ % source /Users/kevinzhao/miniconda/bin/activate base
```

- 在终端中加载 datascience 虚拟环境

```
(base) ~ % conda activate datascience
```

- 启动 Jupyter Notebook

```
(datascience) ~ % jupyter notebook
```

启动 Jupyter Notebook

Starting Jupyter Notebook service



```
kevinzhao — jupyter-notebook — 160x48
Last login: Wed Sep 13 14:13:59 on ttys002
kevinzhao@Kevins-Mac-Studio ~ % source /Users/kevinzhao/miniconda/bin/activate base
(base) kevinzhao@Kevins-Mac-Studio ~ % jupyter notebook

[Outdated]

Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions.
https://jupyter-notebook.readthedocs.io/en/latest/migrate_to_notebook7.html

Please note that updating to Notebook 7 might break some of your extensions.

[I 17:00:55.447 NotebookApp] The port 8888 is already in use, trying another port.
[I 17:00:55.449 NotebookApp] Serving notebooks from local directory: /Users/kevinzhao
[I 17:00:55.449 NotebookApp] Jupyter Notebook 6.5.3 is running at:
[I 17:00:55.449 NotebookApp] http://localhost:8889/?token=aa4ced91cf0684d3550b43cb985519b8f0bd67ad22606a09
[I 17:00:55.449 NotebookApp] or http://127.0.0.1:8889/?token=aa4ced91cf0684d3550b43cb985519b8f0bd67ad22606a09
[I 17:00:55.449 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 17:00:55.451 NotebookApp]

To access the notebook, open this file in a browser:
file:///Users/kevinzhao/Library/Jupyter/runtime/nbserver-28860-open.html
Or copy and paste one of these URLs:
http://localhost:8889/?token=aa4ced91cf0684d3550b43cb985519b8f0bd67ad22606a09
or http://127.0.0.1:8889/?token=aa4ced91cf0684d3550b43cb985519b8f0bd67ad22606a09
[W 17:00:59.632 NotebookApp] 404 GET /apple-touch-icon-precomposed.png (:::1) 13.640000ms referer=None
[W 17:00:59.633 NotebookApp] 404 GET /apple-touch-icon.png (:::1) 14.330000ms referer=None
```

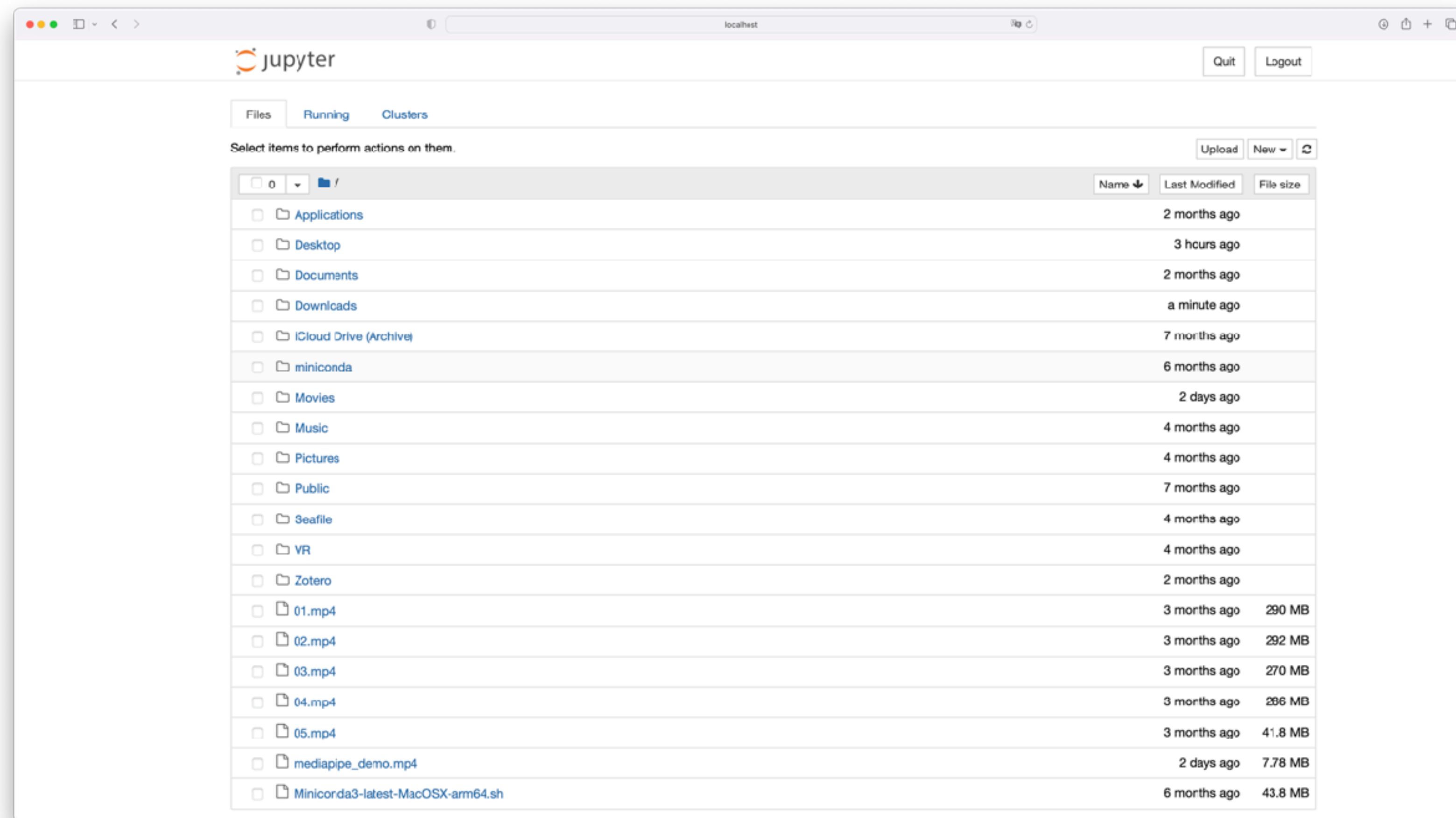
用Ctrl+C可以停止服务

如果浏览器没有自动弹出
就复制这里的地址到浏览器的
地址栏

启动Jupyter Notebook时终端显示

启动 Jupyter Notebook

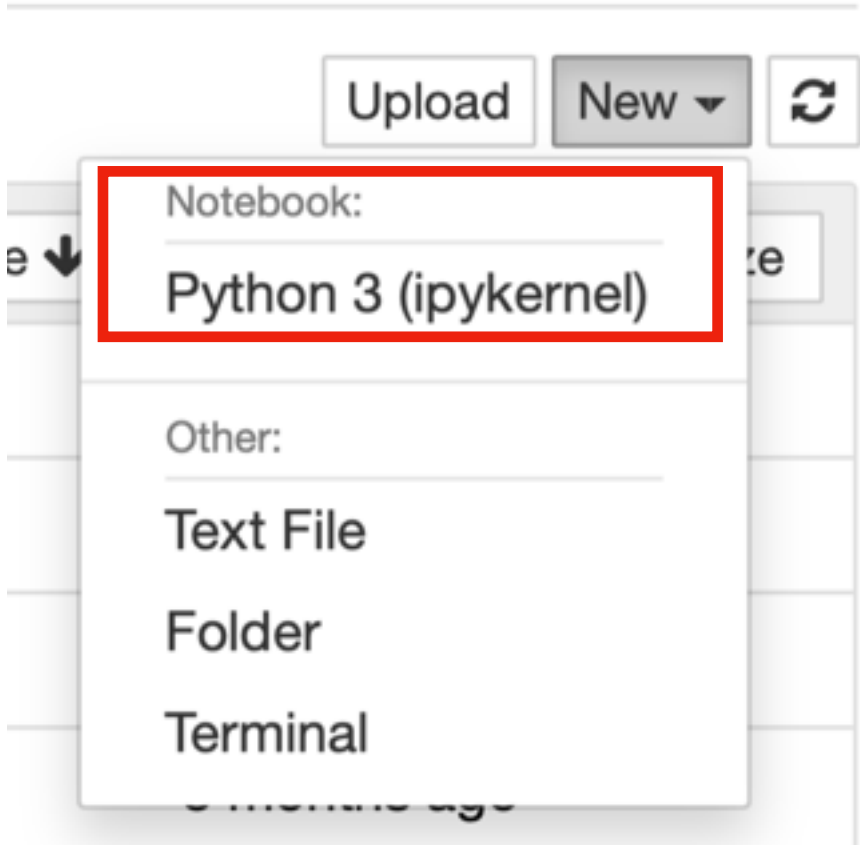
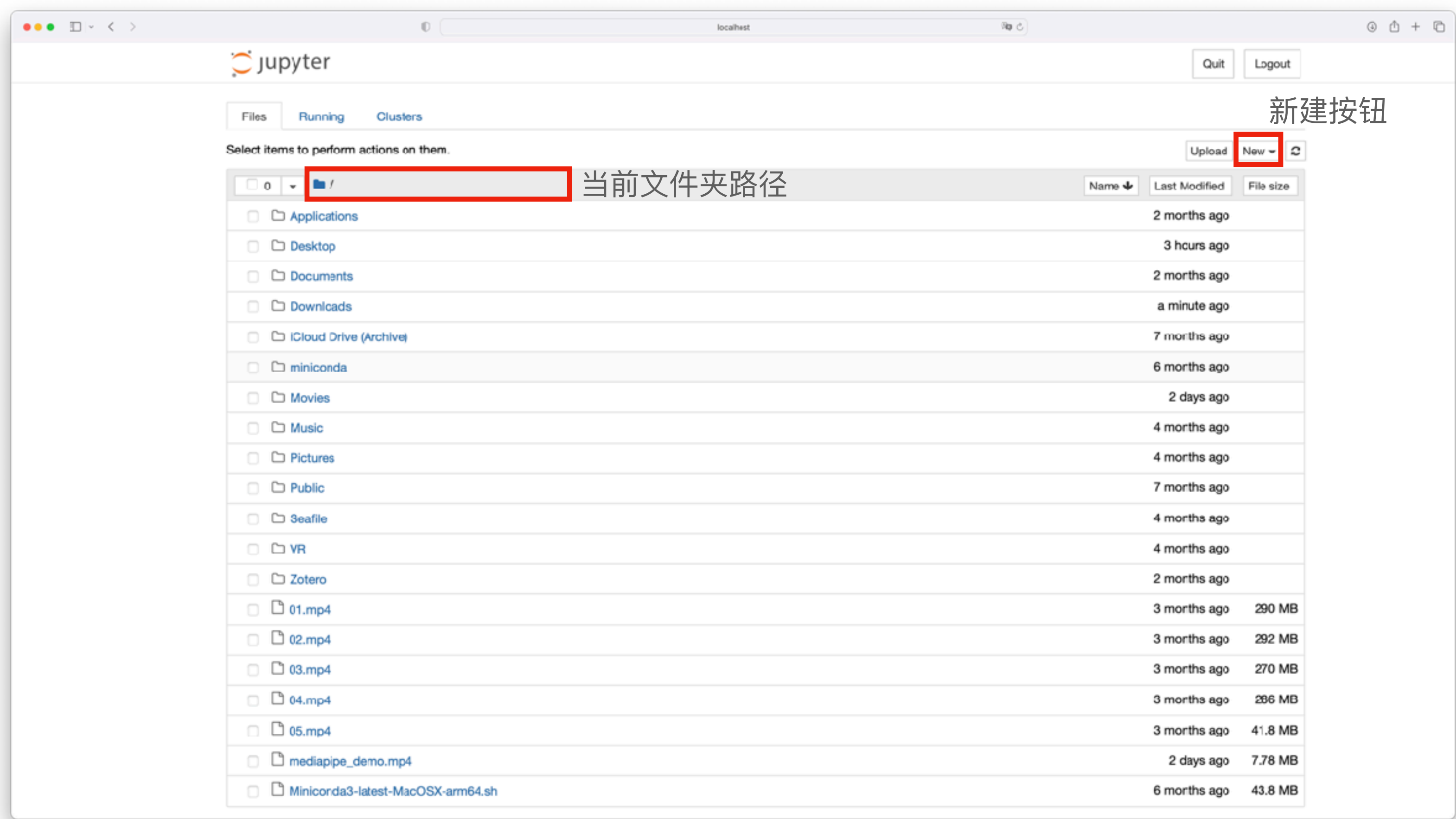
Starting Jupyter Notebook service



Jupyter Notebook 界面

新建 Jupyter Notebook 文档

Create a new Jupyter Notebook file



新建 Notebook

文本文件
文件夹
终端

新建 Jupyter Notebook 文档

Create a new Jupyter Notebook file



文件名

jupyter

Untitled

Last Checkpoint: a few seconds ago (unsaved changes)

kernel 信息

Logout

File

Edit

View

Insert

Cell

Kernel

Widgets

Help

菜单栏

Kernel starting, please wait...

Trusted

Python 3 (ipykernel)

快捷图标

代码框

In []:

代码框 (选中)

In []:

Jupyter Notebook 文档界面

操作 Jupyter Notebook 文档



Operate a Jupyter Notebook file

- Jupyter Notebook有两种模式：
 - 命令模式：按 **esc** 键进入或鼠标直接点击空白处进入，可以对块（block）进行操作。
 - 编辑模式：按 **enter** 键或鼠标直接点击块进入，可以对块进行编辑。

操作 Jupyter Notebook 文档



Operate a Jupyter Notebook file

- Jupyter Notebook有许多快捷键：
 - A 在此块之上添加一个代码块
 - B 在此块之下添加一个代码块
 - DD 删除此代码块
 - X 剪切此代码块
 - M 转换为 Markdown 代码块
 - Y 转换为 Python 代码块
 - L 打开/关闭行号
- ctrl + S 保存
- shift + enter / ctrl + enter 运行此块
- H 打开快捷键帮助

算术操作、变量、 数据类型与逻辑操作

Arithmetical operation, variable, data types and logical operation



算数操作

Arithmetical operation

- Python 支持四则运算以及其他类型的运算操作。

符号	含义	符号	含义
+	加法	**	幂
-	减法	%	取余
*	乘法		
/	除法		

变量

Variable

- 我们可以将运算结果存储到变量中，这样就可以在之后继续使用。

```
foo = 57  
bar = 43 + 60 + 16 + 41
```

foo bar

57

160

```
3 * foo + 5    #read the value from the variable "foo"
```

176

- # 可以注释掉该行后的内容，可以解释代码。

变量

Variable

- 变量名可以是任意长度，可以包含大小写字母A-Z a-z，数字0-9和下划线（_），但是不能以数字开头；
- 变量名大小写敏感，所以age和Age是两个不同的变量；
- 变量名中不能包含特殊字符，如@、：、！、#等；
- 变量名不要使用Python的关键字(keywords)。

```
#导入keyword 模块
import keyword
#显示所有关键字
keyword.kwlist
```

- 由于 Python 3 支持 UTF-8 字符集，因此 Python 3 的标识符可以使用 UTF-8 所能表示的多种语言的字符。

数据类型

Data types in Python

- Python中有多种数据类型：
- 基本数据类型：
 - 数值类型：
 - 整型 **int**
 - 浮点型 **float**
 - 布尔型 **bool**
 - 字符串 **string**
- 高级数据类型：
 - 列表 **list**
 - 元组 **tuple**
 - 字典 **dictionary**
- 使用 `type()` 函数可以方便快捷的确定数据类型：

```
album_year = 1990  
type(album_year)
```

int

字符串

string

- Python中用一对双引号 (") 或者单引号 (') 括住的就是字符串。

```
album = "Thriller"  
b = "1024"  
e = '256'  
type(b)
```

string

逻辑操作

Logical operation

- 布尔值可以用于表示一个条件是成立(为真, **True**)还是不成立(为假, **False**)。

```
album_year = 1990
album_year > 1980 and album_year < 2000
```

True

Python 中的比较运算

符号	含义	符号	含义
>	判断大于	==	判断等于
<	判断小于	!=	判断不等于
>=	判断大于等于		
<=	判断小于等于		

列表、元祖与字典

list, tuple and dictionary



列表

list

list a

3	1	41	5	92	65
---	---	----	---	----	----

- 如果我们要创建一个专辑(albums)列表，那么可以把这些专辑放进一个Python **list**中，这样我们就可以执行各种列表操作了，比如使用**sort()**进行排序。
- 列表的每个元素都有索引（index，序号）：第一个元素是0，第二个元素是1，依次类推。
- 列表有如下特性：
 - 用途最为广泛的数据类型
 - 元素由逗号分隔，整个列表由中括号包围([])。
 - 其中的元素可以用切片操作符 ([]和[:]) 来访问
 - 列表的索引从0开始
 - 加号是列表连接操作符
 - 一个列表可以由多个不同的对象组成，比如向量、数值、字符甚至其他列表。你可以把列表当成一个由关联信息构成的，且结构良好易于读取的容器。

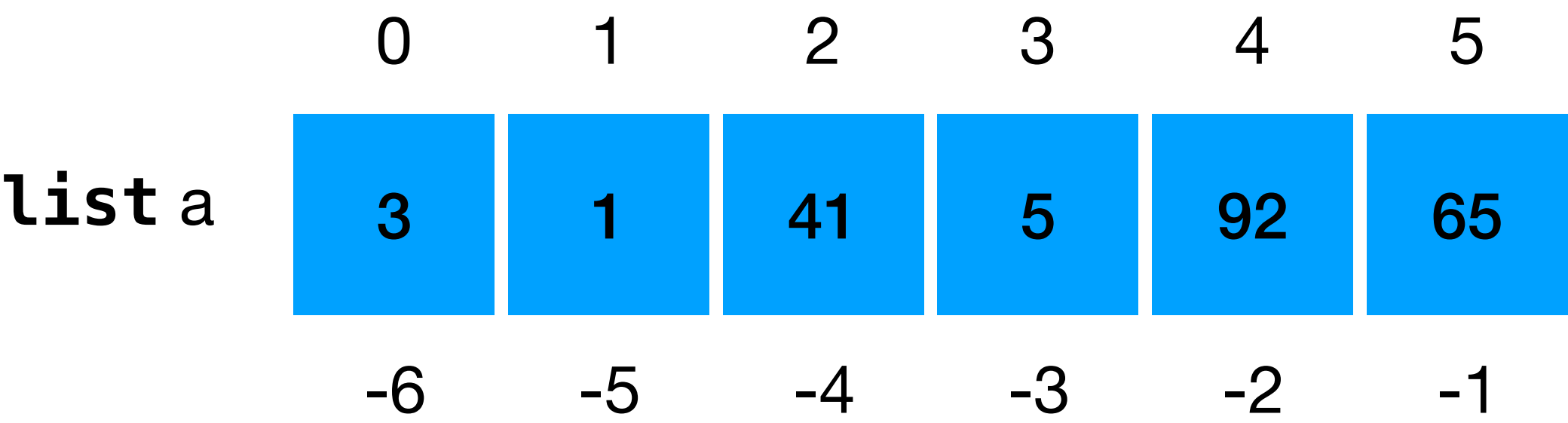
列表

list

操作	使用到的运算符或函数	示例
新建列表变量	= 与 []	Released_dates = [1982,1980,1973,1992]
连接列表	+	a3 = album_titles1 + album_titles2
添加新的元素	append()	Released_dates.append(1960)
获取列表长度	len()	len(Released_dates)
求列表子集	[index], [:]切片	first_element = Released_dates[0] first_few_elements = Released_dates[:3]
列表排序	sort()	Released_dates.sort()
最大值与最小值	min(), max()	

列表中的切片

Slice in list



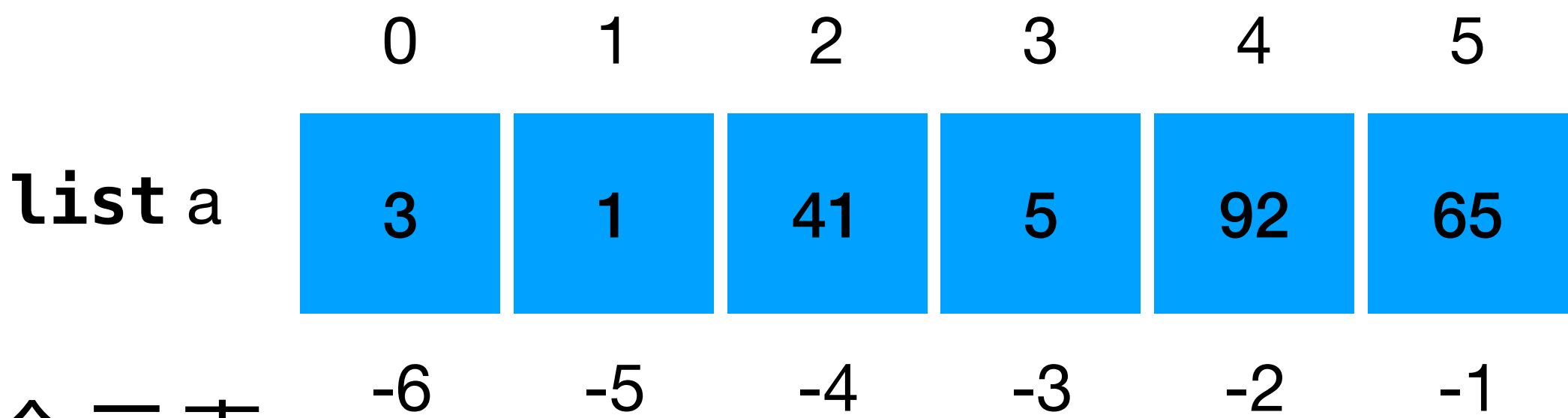
操作	使用到的运算符或函数	示例
取列表中的一个元素	[idx]	dates [0]

- 切片用来获取列表中的片段，即原始列表的子列表。切片用被方括号“[]”括起来且用冒号“:”分隔的数字来表示。
- 注意：切片包含索引项的起始值，而不包含结束值。切片fruits[0:3]获取的是fruits列表的前3项(第0、1、2项)。

操作	使用到的运算符或函数	示例
取列表中的切片	[a:b]	dates [0:3]

列表中的切片

Slice in list



- 我们可以用索引值获取前几个元素或者后几个元素。

操作	使用到的运算符或函数	示例
取列表中的前几个元素	[: x]	dates [: 3]
取列表中的后几个元素	[y :]	dates [5 :]

- 负的索引值会从相对于末尾的位置切片。

操作	使用到的运算符或函数	示例
取列表中的后几个元素	[-w :]	dates [-2 :]

- 我们还可以在第二个冒号后面设置步长。

操作	使用到的运算符或函数	示例
取列表中的几个元素	[:: x]	dates [:: 2]

元组

tuple

- 元组与列表很类似，区别在于元组中的元素不能被修改，并且使用的是圆括号而不是方括号。元组也是用索引机制来访问的。

```
a = (1,1,4,5,1,4)
```

- 但是你**无法对元组排序**，因为它们是不可修改的。解决办法是把**元组转化为列表**，然后进行排序。
- 因为你不能修改元组，所以任何**添加、删除、修改**的操作都只能先转成列表再执行。

```
a_list = list(a)  
a_list.sort()
```

字典

dictionary

- 字典是Python内置的**无序数据结构**。
- 字典如其名所示，其格式为 **(关键字key : 值value)**，我们称其为一个**键值对**。
- 在字典中，任何一个元素都有一个关键字key和值value。元素在字典中没有序号，即没有任何顺序。同时，每个key必须是唯一的。

```
phonebook = {'Andrew Parson':8806336,  
             'Emily Everett':6784346,  
             'Peter Power':7658344,  
             'Lewis Lane':1122345}  
a = phonebook['Andrew Parson']  
a
```

8806336

字符串操作 string operation



字符串操作

string operation

操作	使用到的运算符或函数	示例
获取字符个数	<code>len()</code>	<code>len(summary)</code>
字符串大写	<code>upper()</code>	<code>summary.upper()</code>
字符串小写	<code>lower()</code>	<code>summary.lower()</code>
替换	<code>replace()</code>	<code>summary.replace(" ", "-")</code>
拆分	<code>split()</code>	<code>summary.split()</code>
字符串单词排序	<code>sort()</code>	<code>word_list.sort()</code>
删除头尾字符	<code>strip()</code>	<code>sample.strip(" ")</code>

控制语句：条件与循环

Control statement: Condition statement and loop statement



控制语句

Control statement

- 控制语句是程序员控制哪一段代码可以执行的方法。其语法和正常英文相似，和做逻辑判断一样。
- 条件和循环是改变代码执行流的语句。
- 原本期望的执行流是每一行的代码都会按照从上到下的顺序执行，但是控制语句可以改变语句的执行顺序、跳过一些语句或者重复执行某些语句。

条件语句

Conditional statement

- 我们经常需要去检查一个条件语句的结果，然后根据是对是错来决定接下来执行不同的代码。

Python 中的比较运算

符号	含义	符号	含义
>	判断大于	==	判断等于
<	判断小于	!=	判断不等于
>=	判断大于等于		
<=	判断小于等于		

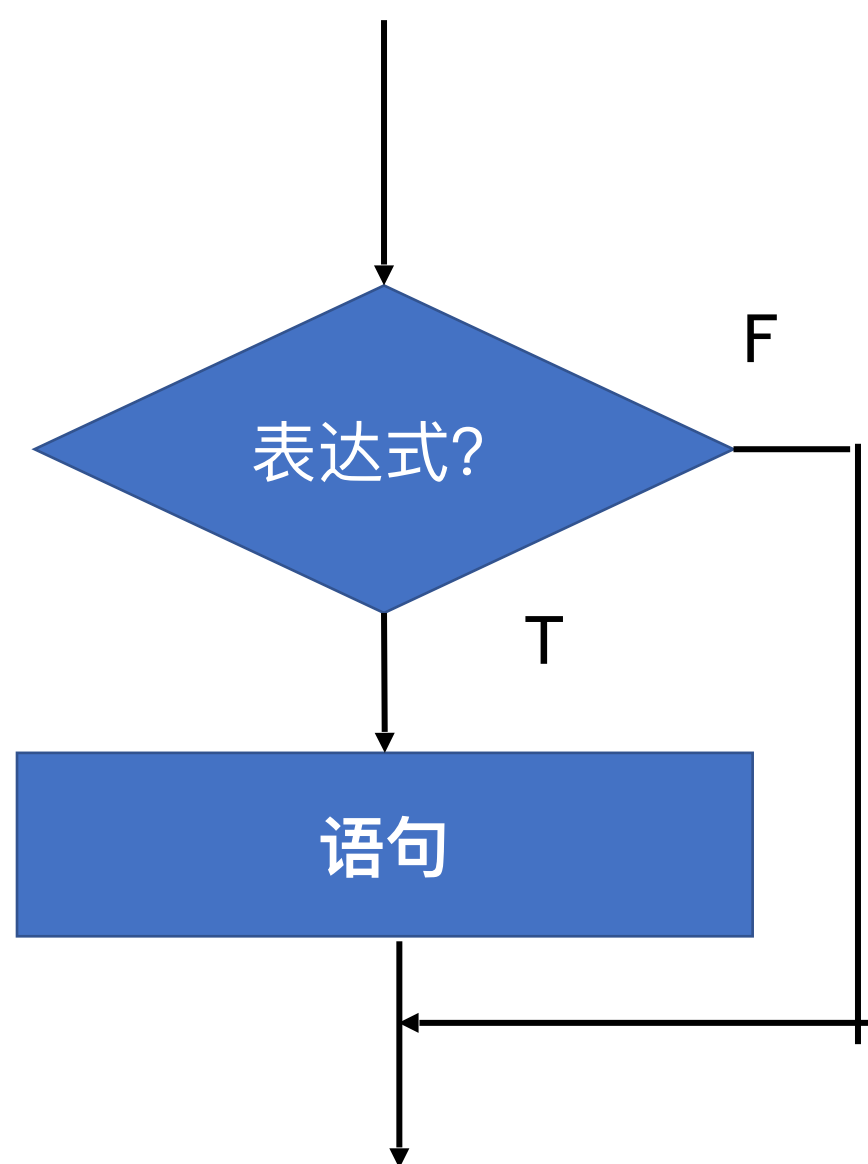
Python 逻辑运算真值表

p	q	与运算 p and q	或运算 p or q	非运算 not p
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

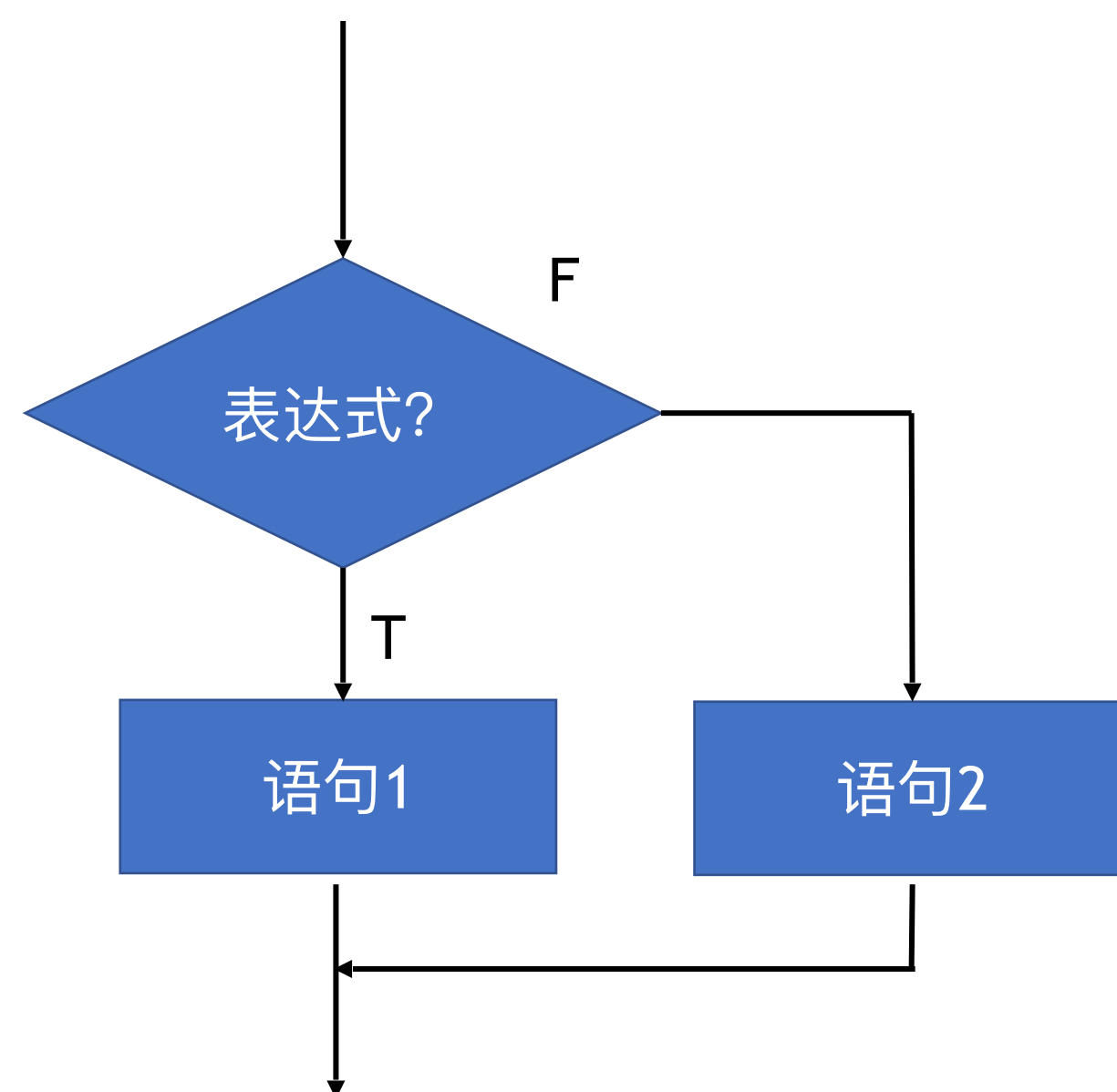
条件语句

Conditional statement

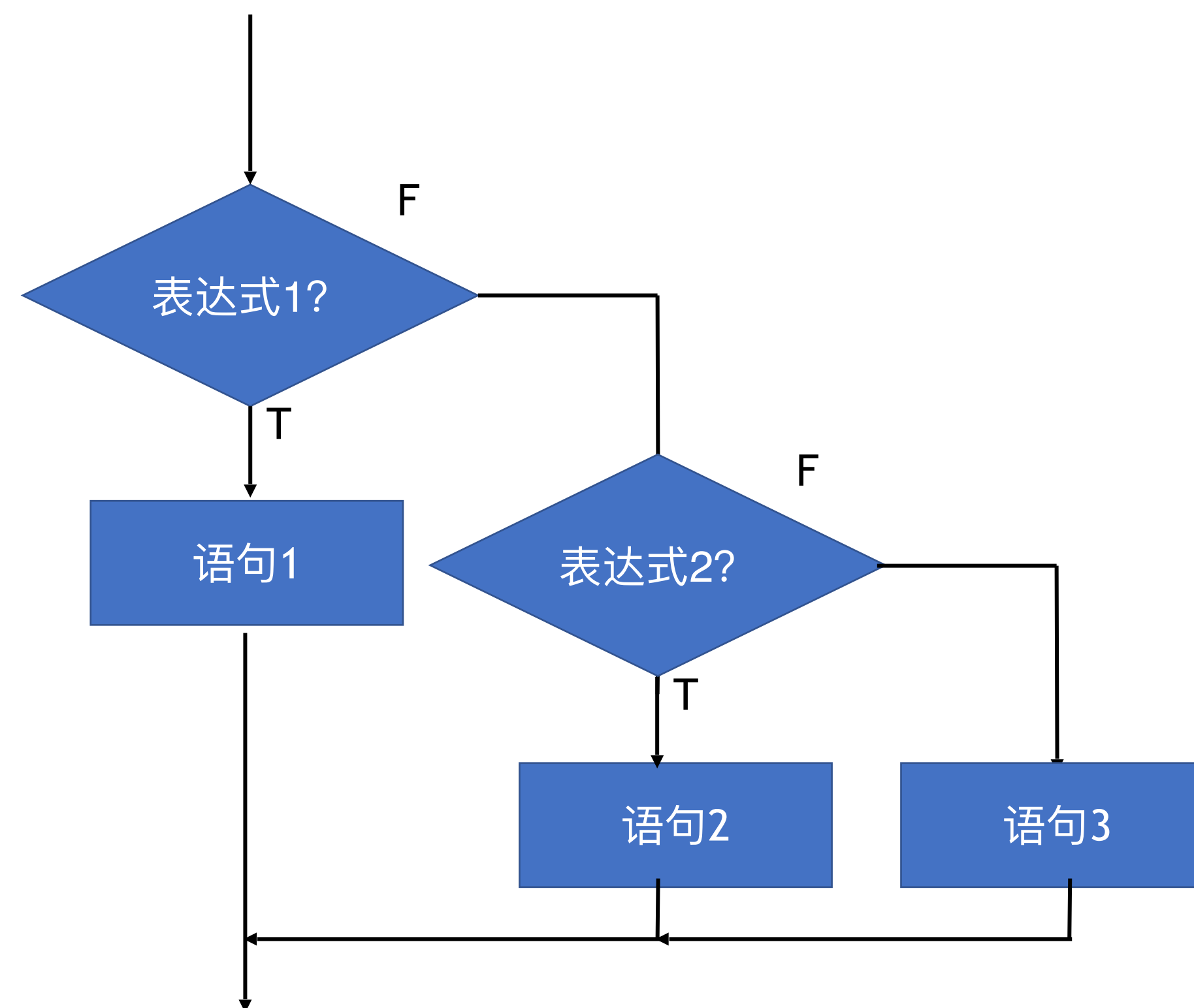
if语句



if-else 语句



if-elif-else 语句



循环语句

Loop statement

- 有时候，你可能希望让一些代码块重复地执行几轮，甚至有可能不知道一共要执行几次，但是有一个循环如何终止的概念。
- 例如：每次一行地处理数据集中的所有行，这种让代码重复执行的方式就是循环。
- 其中有两个主要的循环结构：for 和 while。

循环语句 - for

Loop statement - for

- For循环结构允许你为一个数据结构中的每一个元素执行一次代码块，换一种说法就是：可以遍历数据集中每一个元素。
- 比如说，你想要做到“对数据集中的每一行执行一次处理”或者“对某一系列中所有大于10的元素的每一个都执行一次处理”。for循环是十分实用的语句，它可以对大量的数据进行简单的处理。

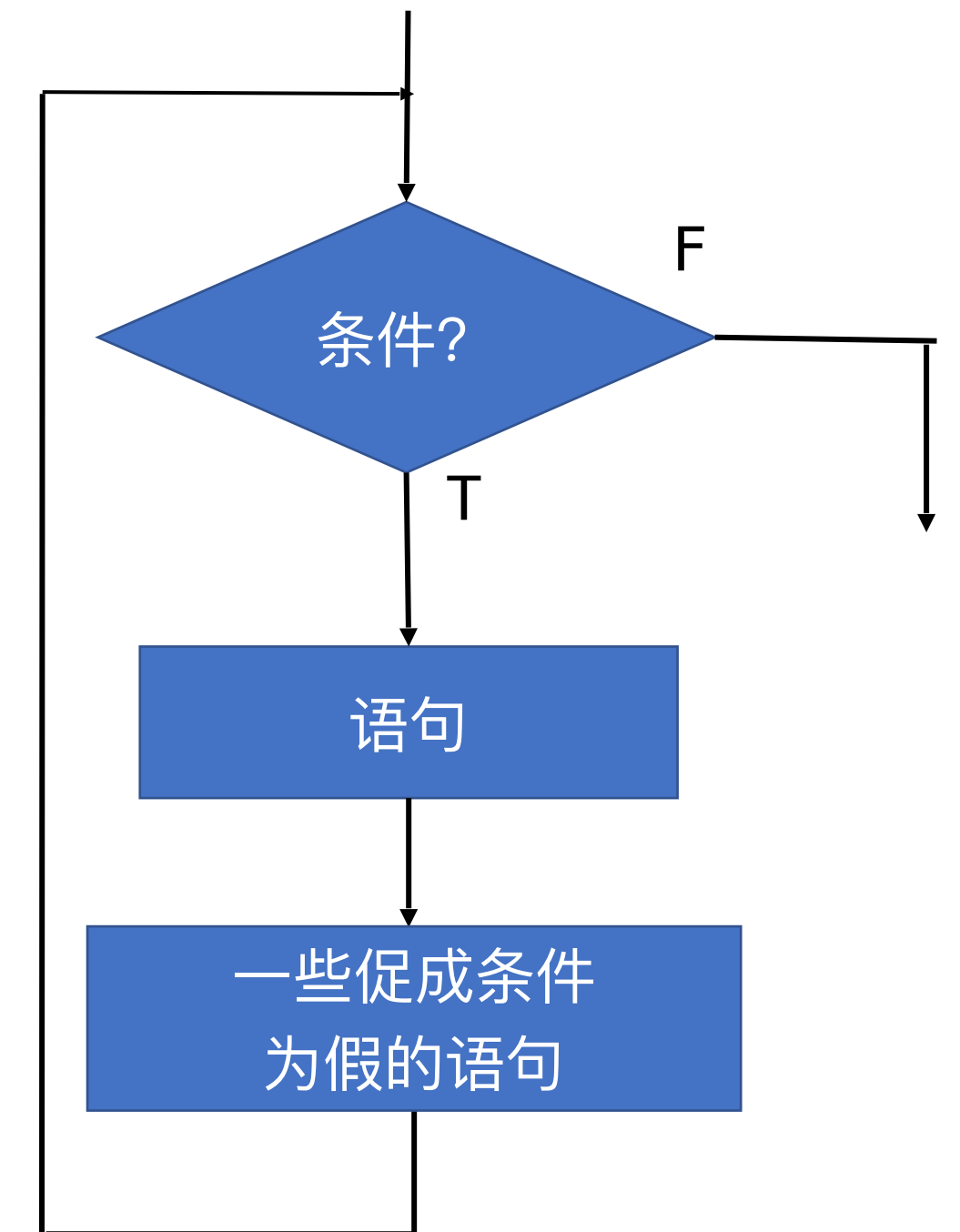
```
released_dates = [1982,1980,1973,1992]
year = 0
for year in released_dates:
    print(year)
```

1982
1980
1973
1992

循环语句 - while

Loop statement - while

- 当我们不知道一个循环应该何时结束的时候，for循环就不能很好地工作了。例如，我们要重复执行一个操作直到某个数值达到了阈值，或者循环终止的条件是一个表达式的结果。
- while循环的结束取决于一个条件是否成立，也就是说循环内的代码段会不断重复执行直到逻辑条件不满足（返回值为False）。



机器学习与深度学习基础

函数 Function



函数

Function

- 函数是一个具有特定功能的可重复使用的**代码块**。
- 共有两种类型的函数：
 - 预定义函数；
 - 用户定义函数。



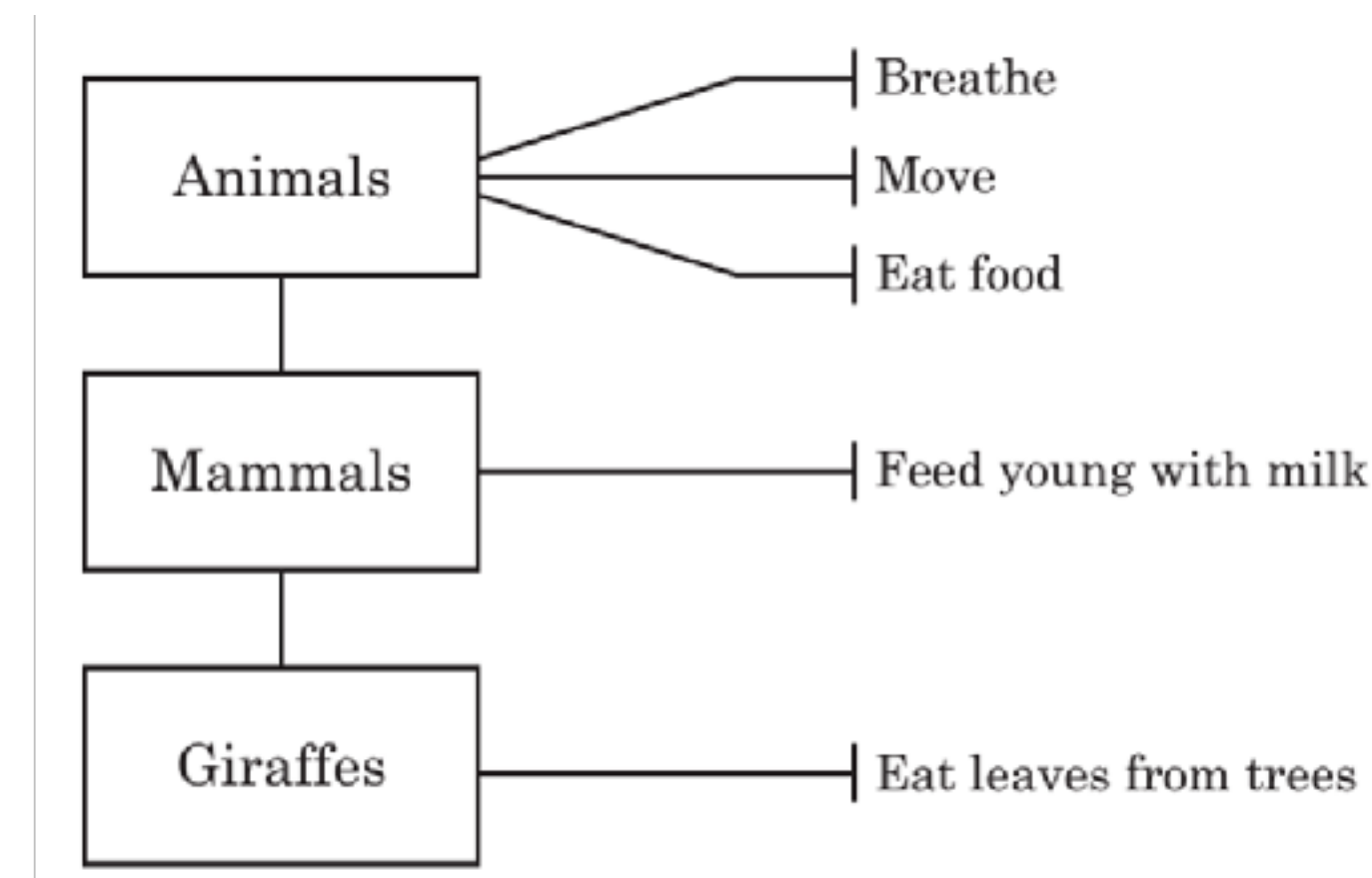
类与对象 Class and object



类

Class

- 类是具有**相同属性和方法**的一组对象的抽象。
- 类为属于该类的所有对象提供了统一的抽象描述。



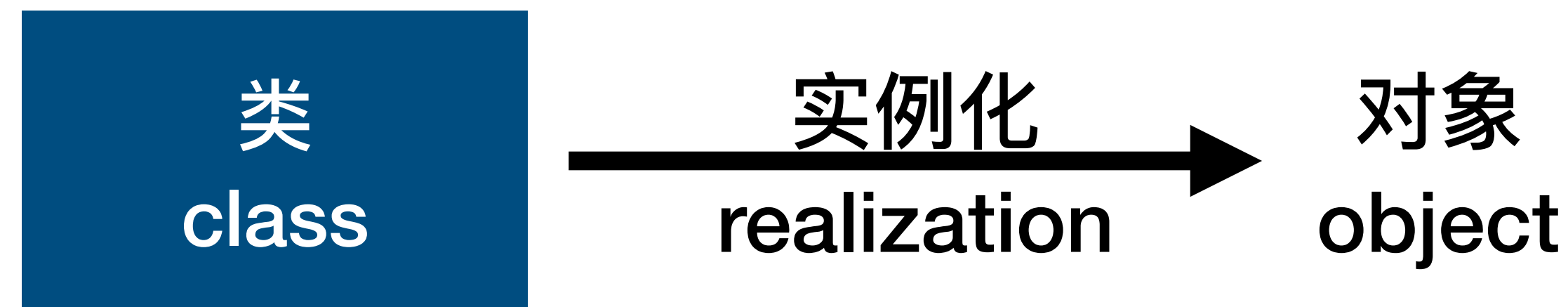
```
class Student:
    def __init__(self, name, age = 18):
        self.name = name
        self.age = age
        self.school = "SJTU"
    def __str__(self):
        printStr = "Student Name: " + self.name + "\n Student Age: " \
        + str(self.age) + "\n Student School: " + self.school
        return printStr
```

- **self** 可以理解作为一种惯例，或者理解为一个**未来的对象**，用来为方法指定某个具体实例。
- 类中所定义的函数称为方法，**self** 作为每个方法的第一个参数列出。
- 函数**__init__** 是初始化对象值的函数，**__str__** 用来返回对象的字符串表达式。

对象

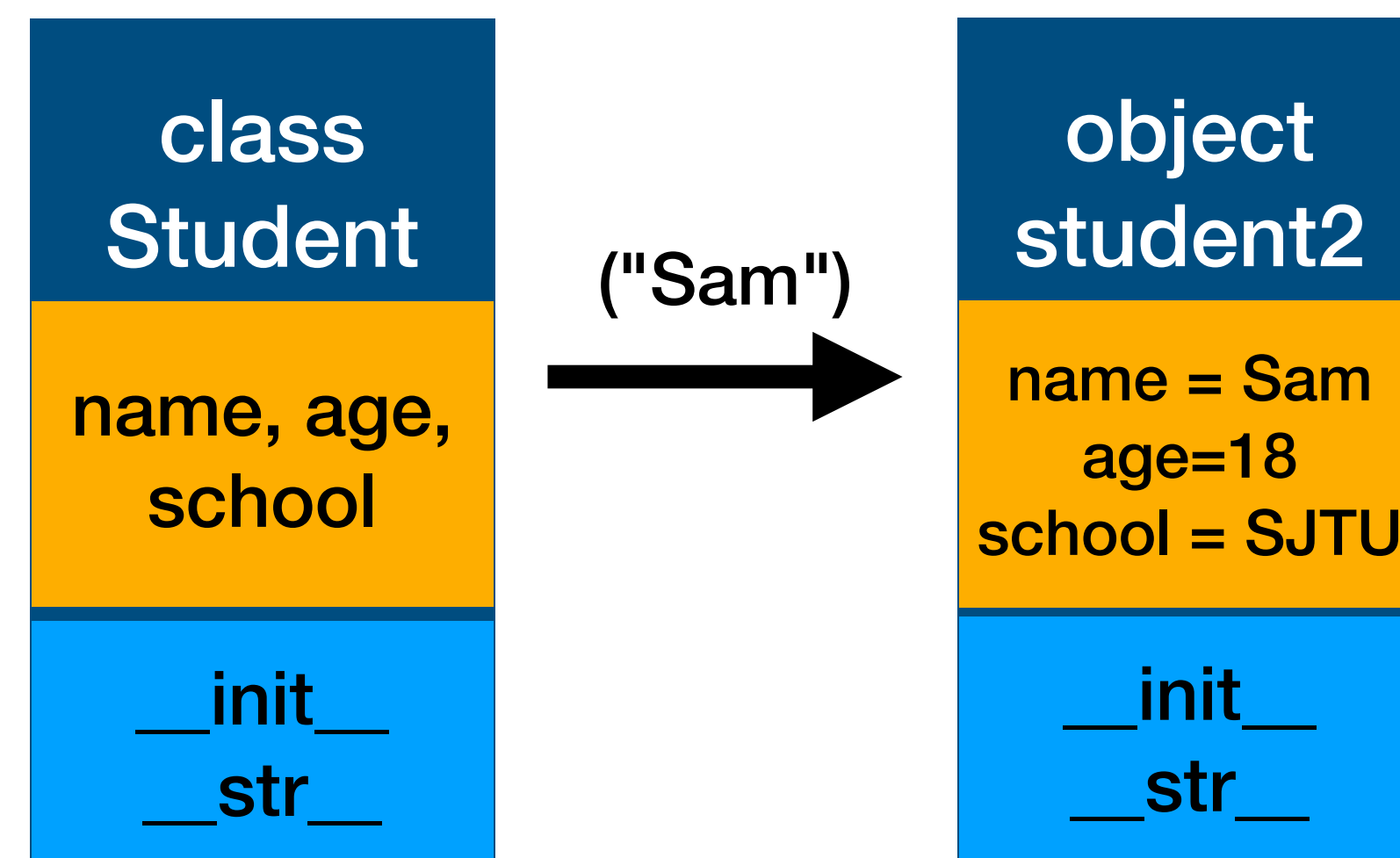
Object

- 对象是系统中用来描述客观事物的一个实体，是构成系统的一个基本单位。
- 对类进行实例化我们就可以得到对象。



```
student2 = Student("Sam")  
print(student2)
```

Student Name: Sam
Student Age: 18
Student School: SJTU



继承

Inheritance

- 继承：从其他已定义好的类中继承其属性。

```
class Student:
    def __init__(self, name, age = 18):
        self.name = name
        self.age = age
        self.school = "SJTU"

class StudentFDU(Student):
    def __init__(self, name, age = 18):
        Student.__init__(self, name, age)
```

```
stu1 = StudentFDU("Liming", 20)
stu1.school
```

SJTU

多态

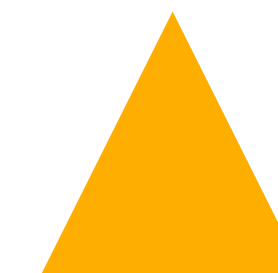
Polymorphism

- 多态：在不同类中使用同一种函数，但为函数定义不同的方法。

```
class Triangle:
    def __init__(self, base, height):
        self.base = base
        self.height = height
    def getArea(self):
        area = 0.5 * self.base * self.height
        return area
class Quadrilateral:
    def __init__(self, long, short):
        self.long = long
        self.short = short
    def getArea(self):
        area = self.long * self.short
        return area
```

```
t1 = Triangle(2,4)
t1.getArea()
```

4.0



```
t2 = Quadrilateral(2,4)
t2.getArea()
```

8



import 导入

import by import

- 我们可以使用 `import` 来导入各种模块或模块内的一部分。
- 可以通过 `import 文件名.类名` 导入其他文件的自建类。

谢谢 Thank you.



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

本幻灯片及其内容（包括但不限于文字、图片、图表、视频、代码和设计）受版权法保护，版权归上海交通大学学生创新中心以及其持有人所有，除明确标注的第三方内容外，未经许可不得转载、修改或用于商业用途。

This presentation and its content, including but not limited to text, images, charts, videos, code, and designs, are protected by copyright laws. The copyright belongs to the Student Innovation Center of Shanghai Jiao Tong University and its respective holders, except for explicitly marked third-party content. . Unauthorized reproduction, modification, or use for commercial purposes is strictly prohibited.